



Figure 1: MAT wizard

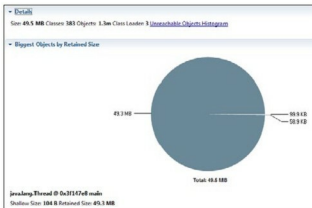


Figure 2: MAT overview

in MAT. The techniques listed in that first article are especially useful in case of IBM JVMs, where the dump provider may not be able to detect a locally running JVM. MAT can also be used to analyse heap dumps that are automatically thrown when the JVM exits due to an *OutOfMemoryError*.

## Analysing dumps

Choose *File* → *Open Heap Dump* to open a heap dump file. This brings up a dialogue box that gives options to analyse the heap dump. The commonly used option is *Leak Suspects Report*.

Choosing the leak suspects brings up the *Overview* page that gives the overall heap information such as the heap size and the number of classes. It also reports the biggest objects in terms of memory consumption. These objects are usually a good place to start the analysis of any memory issues.

The left pane of MAT, by default, shows the object inspector view. This has two sections and is constantly available in most of the views associated with viewing objects. The top part of this view gives the object meta-data such as:

- Memory address of the object
- Class and package names

- Parent object
  - Memory address of the class object and class loader object
  - GC roots of the object
- The bottom part gives object values such as:
- Static members of the object
  - Instance variables
  - Class hierarchy
  - Object value. This is not applicable for most objects, except objects such as *char* arrays.

In order to see the biggest objects, the 'dominator tree' view is very useful. Dominator tree is a term used in graph theory to represent a special type of parent node in a tree. A parent node 'x' is a dominating node of child node 'y' if, from the root of the tree, all traversal paths to child node 'y' go through the parent node 'x'. The same concept applies to heap analysis, since objects present on the heap can be represented as a graph due to references from one object to another. The objects represented as a dominator tree view give information about the 'containing' objects of a particular object or the 'child' objects of a particular object. This helps in identifying the biggest objects that consume memory from a GC root.

In order to understand the concept of the dominator tree better, consider the following code segment:

```
ArrayList<MyBigObject> list = new ArrayList<MyBigObject>();
for (int i = 0; i < 99999; i++) {
    MyBigObjectobj = new MyBigObject();
    list.add(obj);
}
```

In the above code, the *ArrayList* object 'list' is the dominator object of all the *MyBigObject* instances created. So, in MAT, all the instances of *MyBigObject* can be viewed by expanding the 'list' object. The retained size of *ArrayList* will include the retained size of all instances of *MyBigObject*. On the other hand, consider the following code segment:

```
ArrayList<MyBigObject> list = new ArrayList<MyBigObject>();
Map<Integer, MyBigObject> map = new HashMap<Integer,
MyBigObject>();
for (int i = 0; i < 99999; i++) {
    MyBigObjectobj = new MyBigObject();
    list.add(obj);
    map.put(i, obj);
}
```

In the above code, 'list' is no longer the dominating object for all the instances of *MyBigObject*. This is because these instances are now accessible through another path 'map' as well. Hence, in the dominator tree, the retained size of 'list' does not include the retained sizes of instances of *MyBigObject*. It only contains the memory occupied by the references to these instances. This is a very important